

* Analysis of Algorithm.

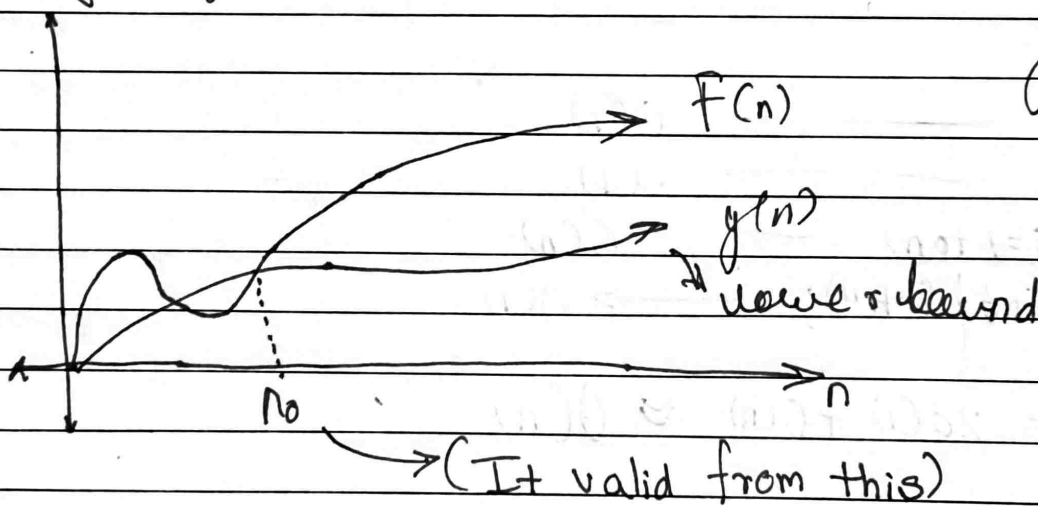
i) Big Oh - O notation.

Where C is constant
 $[C > 0]$

$g(n) \rightarrow$ upper bound function.

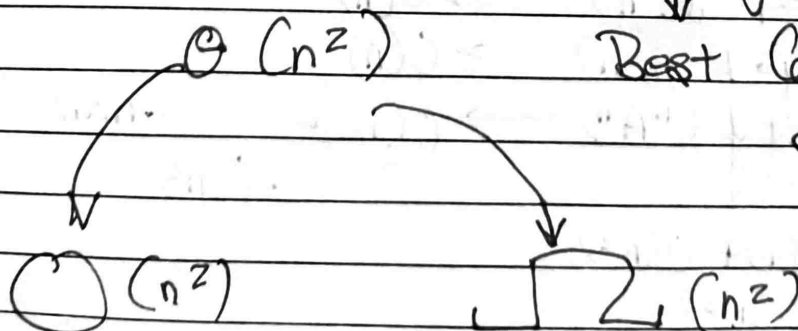
$f(n) \rightarrow$ Running time of an Algorithm.

ii) Big Omega Ω (Best case T.C) $O(n^2)$



iii) Theta $\Theta \rightarrow$ Average Case.

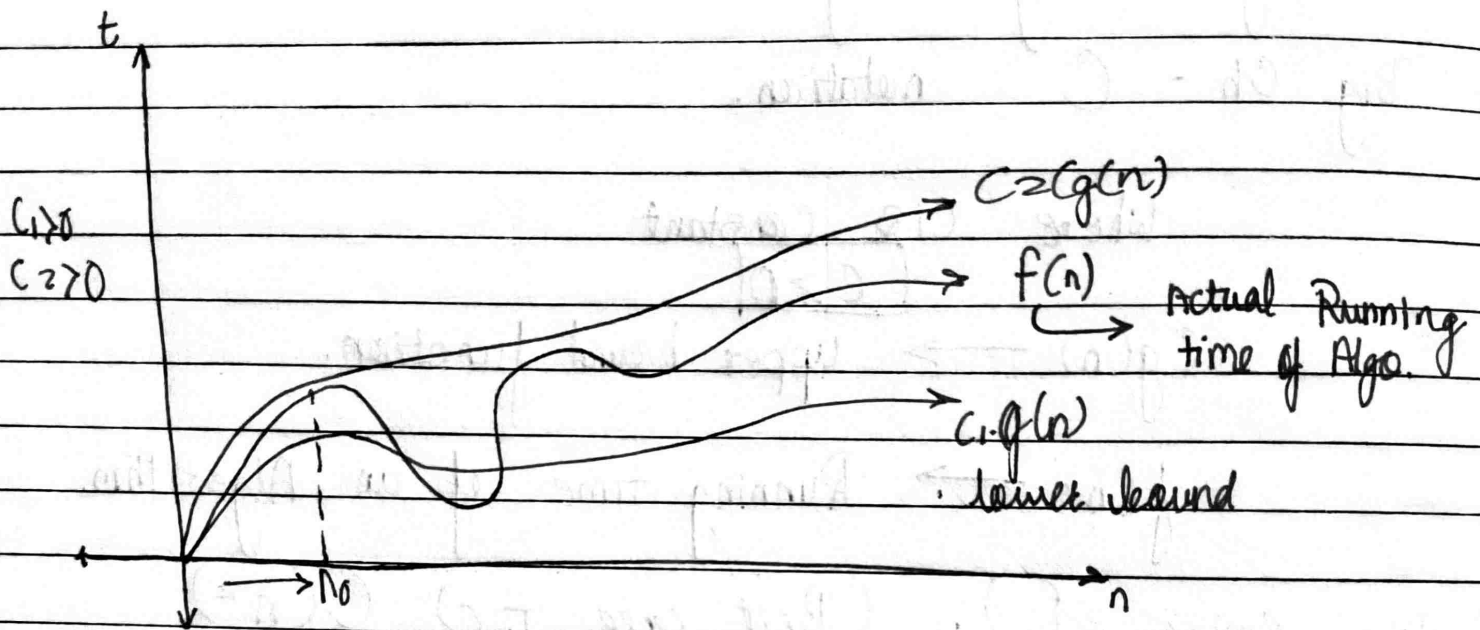
$\downarrow$
 Best Case & Worst Case
 are same.



$$O(k \cdot n) \approx O(n)$$

Page No. _____

Date ____/____/____



ex. A()

```

6 int i          _____ O(1)
  int i;         _____ O(1)
  for(i=1 to n)  _____ O(n)
    printf("A"); _____ O(1)
  
```

$$T.C = 2O(1) + O(n) \approx O(n)$$

ex. 2) A()

```

{ int i, j;      _____ O(1)
  for (i=1 to n) _____ O(n)
    for (j=1 to n) _____ O(n)
      printf("A") _____ O(1)
    }
  }
  
```

$n \times n$
 $O(n^2)$

$$T.C = O(1) + O(n^2)$$

$$= O(n^2)$$

ex: 3) A()

{ i = 1 $\longrightarrow$ O(1)for (i = 1, i * i $\leq$ n ; i++){ print("A") $\longrightarrow$ O($\sqrt{n}$)
}i = 1
i = 2
...i² $\leq$ ni $\leq$ $\sqrt{n}$ $\sqrt{n}$ timesT.C = O($\sqrt{n}$)

ex. 4) A()

{ int i;

int j;

for (i = 1; i $\leq$ n; i++) $\longrightarrow$ "n" times{ for (j = 1; j $\leq$ i; j++) $\longrightarrow$ "i" times{ for (k = 1; k $\leq$ 100; k++) $\longrightarrow$ "100" times

{

print("A")

{ } }

unrolling the loop

$i=1$	$i=2$	$i=n$
$j=1 \text{ time}$	$j=2 \text{ time}$	$j=n \text{ times}$
$k=100 \text{ times}$	$k=200 \text{ times}$	$k=n \times 100 \text{ times}$

$$T.C = 100 + 2(100) + 3(100) + \dots + n(100)$$

$$T.C = 100 \cdot n(n+1)/2$$

$$T.C = 50n^2 + 50n$$

$$T.C \approx n^2 + n$$

Note: $f(n) = n^k + n^{k-1} + \dots$
 $= O(n^k)$

Que A()

```

{
  for (i=1; i<=n; i=i*2)
  {
    print ("A.A")
  }
}

```

→ $i = 1, 2, 4, 8, 16, \dots$ $\left(\frac{n}{2^k} \right)$ → terminate pt.
 $2^0, 2^1, 2^2, 2^3, \dots$
 $j = n \rightarrow \text{ter. pt.}$
 $n = 2^k$

$$\log_2 n = \log_2 2^k$$

$$\log_2 n = k \log_2 2$$

$$k = \log_2 n$$

$$\approx O(\log_2 n)$$

Que. $A()$

```

while (n > 1)
{
  n = n/2;
}

```

Assume $n \geq 2$

$n=100$	$n=50$	$n=25$
$A = \frac{100}{2}$	$A = \frac{50}{2}$	$A = \frac{25}{2}$
$A = 50$	$A = 25$	$A = 12.5$

no. of iterations = 4

```

for (n = 100; n > 1; n = n/2)
  for (n = 1; n <= k; A = n * 2)

```

2) Recursive Functions:

```

A(n)
{
  if ( ) ———  $O(1)$ 
  {
    return  $A(\frac{n}{2}) + A(\frac{n}{2})$ 
  }
}

```

$T(n) \rightarrow \text{Time f}^n$
 $\rightarrow O(1)$
 $\rightarrow 2T(\frac{n}{2})$

T.C = $T(n) = O(1) + 2T(\frac{n}{2})$ Rec. eqn:

```

A(n)
{
  if (n > 1)
  {
    return (A(n-1))
  }
}

```

$T(n) \rightarrow \text{Time f}^n$
 $\rightarrow O(1)$
 $\rightarrow T(n-1)$

T.C = $T(n) = O(1) + T(n-1)$: $n > 1$

// R. car.

Base case (B.C.)

$T(n) = O(1)$ | $T(n) = 1$; $n = 1$

// B.C.

* Back - Substitution Method :-

$$\boxed{T(n) = T(n-1) + I} \quad ; n > 1 \quad \text{--- (1)}$$

$$T(n) = I \quad ; n = 1$$

$$\bullet \quad \boxed{\text{For : } T(n-1) = T(n-2) + I} \quad \text{--- (2)}$$

(n-1-1)

$$\bullet \quad \text{For } T(n-2) = T(n-3) + I \quad \text{--- (3)}$$

Eqⁿ (1) becomes

ineqn (1) put $T(n) =$

$$T(n) = [T(n-2) + I] + I$$

$$\text{sub (1) - } T(n) = T(n-2) + 2$$

$$\text{sub (2) - } T(n) = [T(n-3) + I] + 2$$

$$T(n) = T(n-3) + 3$$

⋮

After Substitution 'k' times.

$$\boxed{T(n) = T(n-k) + k}$$

$n-k = 1$ $\xrightarrow{\text{Ter. Pt.}} k = n-1$
 $n-k = 1 \Rightarrow k = n-1$

$$T(n) = T(1) + n-1$$

$$T(n) = I + n - I$$

$$T(n) = n$$

$$= O(n)$$

Que) eqn:-

$$\boxed{T(n) = T(n-1) + n} \quad \text{--- (1) ; } n \geq 1$$

$$T(n) = I \quad ; n = I.$$

$$\begin{aligned} \text{For } (n-1) &= T(n-1-1) + (n-1) \\ &= T(n-2) + (n-1) \quad \text{--- (2)} \end{aligned}$$

$$\begin{aligned} \text{For } (n-2) &= T(n-2-1) + n \\ &= T(n-3) + (n-2) \quad \text{--- (3)} \end{aligned}$$

eqn (1) becomes (after back-Substitution Method) (2)

$$T(n) = [T(n-2) + n(n-1)] + n$$

$$T(n) = T(n-2) + n + n-1$$

$$T(n) = [T(n-3) + (n-2)] + n + n-1$$

$$T(n) = T(n-3) + n + n + n-3$$

After 'k' Substitution:-

$$T(n) = T(n-k) + kn - k.$$

$$T(n) = T(n-k) + k(n-1)$$

terminates when $T(n-k)$ becomes $T(1)$

and $n-k$ becomes I

$$n-k = I$$

$$\boxed{k = n - I}$$

$$T(n) = T(I) + (n-I)(n-1)$$

$$T(n) = (n-1)^2 + O(1)$$

$$\boxed{T(n) = O(n^2)}$$

$$\bullet \rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n ; n > 1$$

$$T(n) = 1$$

$$; n = 1$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \text{ --- (2)}$$

$$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \text{ --- (3)}$$

$$T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n$$

$$T(n) = 2^2 T\left(\frac{n}{2^2}\right) + n + n$$

$$= 2^2 \left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \right] + n + n$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + n + n + n$$

After 'k' Substitutions

$$T(n) = 2^k \left(T\left(\frac{n}{2^k}\right) + k \cdot n \right)$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log_2 n$$

$$T(n) = n^{\log_2 2} + n \log_2 n$$

$$= n' + n \log_2 n$$

$$T(n) = O(n \log n)$$

★ Master's Theorem :

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n^k \log^p n)$$

$a \geq 1$, $b > 1$, $k \geq 0$ and p is real no)

case's

1) if $a > b^k$ then $T(n) = O(n^{\log_b a})$

2) if $a = b^k$

a) if $p > -1$, then $T(n) = O(n^{\log_b a} \log^{p+1} n)$

b) if $p = -1$ then $T(n) = O(n^{\log_b a} \log \log n)$

c) if $p < -1$, then $T(n) = O(n^{\log_b a})$

3) if $a < b^k$

a) if $p \geq 0$, then $T(n) = O(n^k \log^p n)$

b) if $p < 0$, then $T(n) = O(n^k)$

★ Linear Search:

hSearch (Arr, N, X)

{ ... i = 0, found = 0;

while (i < N)

{

if Arr[i] == X

{

print(i)

found = 1;

i++

}

if (found == 0)

print("Not found") ?



Arr	2	4	8	5	0	-1	2	9	5	10
	0	1	2	3	4	5	6	7	8	9

B.C = No. of Steps i. = N-1

B.C = O(N)

W.C = No. of Steps = N-1
= O(N)

Average case = O(N)

* Linear Search on Sorted Array:

```

Lsearch(Arr / N / X)
{
    i = 0, found = 0;
    while (i < N && A[i] <= X)
    {
        if (Arr[i] == X)
        {
            print(i);
            found = 1;
            i++;
        }
    }
    if (found == 0)
        print "Not found"
}

```

B.C = No. of Steps = 1

T.C = $\sum C_i$

W.C = No. of Steps = $N - 1$

T.C = $O(N)$ [No. of Steps $\leq (N - 1)$]
 → for sorted array.

* Binary Search:

```

recurs. b.search (Arr / N / X / lo / hi)
{

```

```

    mid, ret_val;

```

```

    if (low > high)

```

```

        ret_val = -1;

```

```

    else

```

```

    {
        mid =  $\lfloor (lo + hi) / 2 \rfloor$ 

```

```

        if (A[mid] == X)

```

```

        {
            ret_val = mid;

```

```

        }
    }

```

```

else if (A[mid] > x)
{
    retinal = rec-b.search(Arr, N, x, co, mid-1)
}
else
{
    retinal = rec-bsearch(Arr, N, x, mid+1)
}
return retinal.
}

```

* Re-cursive Binary Search. (ADT, low, high, X)

int retrieval, mid;

$$\text{mid} = (\text{low} + \text{high}) / 2;$$

$O(1)$ if $(A[\text{mid}] == x)$
return mid;

$O(1)$ ~~base~~ else if $(A[mid] < x)$ // low = mid + 1, high = same.
 $(\frac{n}{2})$ ~~return~~ = recursive binary search(A , mid + 1, high, x)

$T\left(\frac{n}{2}\right) \leftarrow$ else if $(A[\text{mid}] > X)$ // low = same, high = mid - 1.
 return recursive binary search(A, low, mid - 1, X).

```
else if (low > high) // crossover of array endpoints
    return -1
```

return retval;

ex. Arr

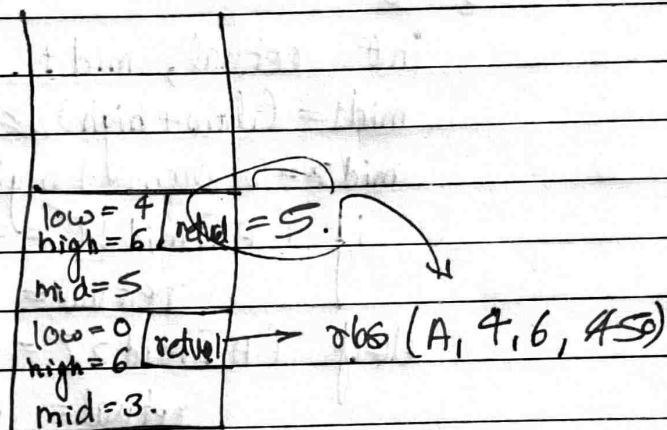
10	20	30	40	45	50	55
----	----	----	----	----	----	----

 0 1 2 3 4 5 6

rbt(A, 0, 6, 45) $\Rightarrow$
 Time complexity $T(n)$
 $T.C \Rightarrow$

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$T(n) = T(n/2) + 1$$



$$T(n) = \left[T\left(\frac{n}{4}\right) + 1 \right] + 1 \quad \text{--- Ist substitution.}$$

$$T(n) = \frac{n}{k} T\left(\frac{n}{4}\right) + 2 \quad \text{--- B.} = T\left(\frac{n}{2^2}\right) + 2$$

$$T(n) = \left[T\left(\frac{n}{8}\right) + 1 \right] + 2 \quad \text{--- 2nd substitution}$$

$$T(n) = T(n/8) + 3 \quad \text{--- C.}$$

⋮

$$T(n) = T(n/2^k) + k \quad \text{--- after } k^{\text{th}} \text{ substitution}$$

Base case $\Rightarrow T(n) = 1 ; n = 1.$

$$\text{If } T\left(\frac{n}{2^k}\right) = T(1) \Rightarrow \frac{n}{2^k} = 1.$$

$$n = 2^k \Rightarrow \log_2 n = \log_2 2^k$$

$$k = \log_2 n \Rightarrow B \Rightarrow T(n) = 1 + \log_2 n$$

$$T(n) = O(\log n).$$

* Tertiary Search.

```

int retval, mid1, mid2;
mid1 = (low + high) / 3;
mid2 = 2 * (low + high) / 3;
if (A[mid1] == x)
    retval = mid1;
elseif (A[mid2] == x)
    retval = mid2;
elseif (A[mid1] > x)
    retval = rbs(A, low, mid1 - 1, x);
elseif (A[mid1] < x && A[mid2] > x)
    retval = rbs(A, mid1 + 1, mid2 - 1, x);
elseif (A[mid2] < x)
    retval = rbs(A, mid2 + 1, high, x);
else if (low > high)
    retval = -1;

```

return retval;

* Trees

- 1) Tree is a collection of vertices (V), set of Edges (E).
- 2) Tree is a connected graph.
- 3) There are no cycles, it is acyclic.
- 4) When we have a tree where every node has k or less than k children.
Then it is called k -ary tree.

5. Maximum number of children it can have is 2 childrens, but leaf node have 0 childrens.
6. Root node doesn't have any incoming edge.
7. Leaf node doesn't have any outgoing edge.
8. If maximum level in the binary tree is k then total number of nodes that can be present equals to $2^{k+1} - 1$.
9. Height is a largest level possible in the given tree.

* Recursive definition of a tree

Tree is a root node + left sub tree + Right sub tree, where LST & RST will be the trees by themselves.

* Implementation of tree.

```

struct tree node
{
    int data;
    tree node * lptr * rptr;
} TN;
  
```

Que. Write a function to count no. of nodes in binary trees.

→ $N(T) = 1 + N(LST) + N(RST)$

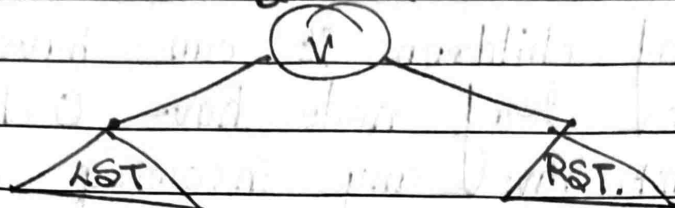
$NT(\text{root}) \{$

if ($\text{root} \rightarrow \text{lptr} == \text{NULL}, \text{root} \rightarrow \text{rptr} == \text{NULL}$)
 " No. of node = 1 "

else

 " no. of node = $1 + NT(\text{root} \rightarrow \text{lptr}) + NT(\text{root} \rightarrow \text{rptr})$ "

* Binary Search Tree.



$$LST \leq V \leq RST$$

Assuming V is a unique key in case different records have the same key i.e. duplicated values are present then for practical purpose we assume V as a unique values.

Tree Search (Tree Node * root, int x)

{ if (root != NULL) {

 If (root == x)

 return root.

 else if (x < root)

 return = Tree Search (Tree Node * root, int x)

 else if (x > root)

 return = Tree Search (Tree Node * root, x)

 else if (Tree Node * root = NULL)

 return = -1

 } return return = ;

* Iterative Binary Search Tree :

Tree Search (root, int x)

{

 O(1) - P = root

 while (P != NULL && key != P)

 if (key < P - data)

 P = P → left.

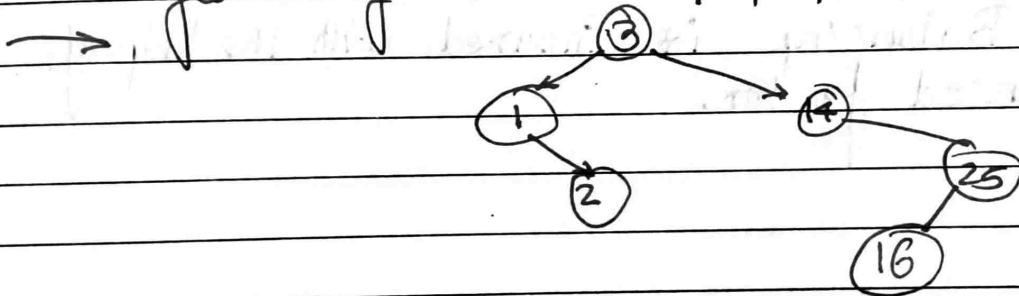
else

$p = p \rightarrow \text{right}.$

return p

}

* Insert the elements in BST in the following order 3, 1, 2, 14, 25, 16.



Insert BST (root, x)

{ if (root == NULL)

root = x

p = root, done = 0

while (!done) {

if (x → data < p → data)

{ if (p → left != NULL)

{ p = p → left.

}

```

    }
    else if (k → data > p → data)
    {
    }
}

```

* Definitions of balanced tree

* 1) If tree is ~~not~~ having equal no. of nodes on each side of root and it should be recursively followed.

2) LST & RST must have equal height for atmost difference of 1. also this should be recursively followed.

* Ex of balanced binary search tree is

* AVL Tree

3) The Balancing is insured with the help of balanced factor.

Balance Factor.

1) If Height of LST = Height of RST then, B.F = 0.

2) If LST Height = 1 + Height of RST then, B.F = 1.

3) If Height of RST = 1 + Height of LST then, B.F = -1.

4) If $-1 \leq B.F \leq 1$ then tree is AVL.